

CUDA and OpenMP Multigrid Solvers for the Poisson Equation in 2D and 3D

Sheng-Chieh Tseng

Department of Physics, National Taiwan University

5 Jun 2026

Project repository:

<https://github.com/TCC0731/multigrid-poisson-solvers>



- ① Problem Formulation
- ② Multigrid Method
- ③ CUDA Multigrid Results
- ④ OpenMP Performance
- ⑤ Conclusions

Governing equation and discretization: Continuous and discrete model

Continuous model

We solve the positive-definite Poisson problem with Dirichlet boundary conditions:

$$\mathcal{L}\phi = f, \quad \mathcal{L} = -\nabla^2,$$

on $\Omega = [0, 1]^d$, with $d \in \{2, 3\}$.

Discrete model

$$h = \frac{1}{n+1}, \quad \phi_0|_{\text{interior}} = 0, \quad \phi|_{\partial\Omega} = \phi_{\text{exact}}|_{\partial\Omega}.$$

The initial guess is zero in the interior, while boundary values are fixed by the manufactured solution. The code uses the standard 5-point stencil in 2D and 7-point stencil in 3D.

Governing equation and discretization: Discrete operator and boundary conditions

Dimensionless discrete operator

The interior unknowns satisfy

$$A_h \phi_h = h^2 f_h, \quad r_h = h^2 f_h - A_h \phi_h.$$

In 2D, the standard 5-point stencil is

$$(A_h \phi)_{i,j} = 4\phi_{i,j} - \phi_{i-1,j} - \phi_{i+1,j} - \phi_{i,j-1} - \phi_{i,j+1},$$

and in 3D the 7-point stencil replaces 4 by 6 and includes the six axis-aligned neighbors.

- Dirichlet boundary data are taken from the manufactured solution; sine gives homogeneous Dirichlet data, while cosine gives inhomogeneous Dirichlet data.
- The arrays store n interior points per axis plus one boundary layer on each side, so the total size is $n + 2$.

Algorithms and backends: Solver and backend support

Implemented solvers

- Red-black SOR baseline.
- Multigrid Poisson solver in 2D and 3D.
- Both V-cycle and W-cycle variants.
- Coarsest-grid solve with SOR or exact/direct solve.

Implemented support

- Python backend.
- OpenMP CPU backend.
- CUDA GPU backend.
- Full-weighting restriction and bilinear/trilinear prolongation.
- Manufactured-solution Dirichlet boundary setup.

Algorithms and backends: Stopping criterion and smoother setup

- Convergence criterion: the relative scaled residual

$$\eta = \frac{\|h^2 f - A_h \phi\|_2}{\|h^2 f\|_2}$$

where A_h is the dimensionless finite-difference stencil approximating $h^2(-\nabla^2)$, must satisfy $\eta \leq \text{tol}$.

- Baseline smoothers: red-black SOR.
- Multigrid smoothing: red-black SOR with ν pre- and post-smoothing steps.
- If $\omega = 1$, the red-black SOR update reduces to red-black Gauss–Seidel.

Automatic SOR weight

When ω is left on auto, the code uses

$$\omega_{\text{auto}} = \frac{2}{1 + \sin(\pi/(n+1))}.$$

Here n is the number of interior points per axis on the current grid level.

- 1 Problem Formulation
- 2 Multigrid Method**
- 3 CUDA Multigrid Results
- 4 OpenMP Performance
- 5 Conclusions

Transfer operators and cycles: Restriction and prolongation

- Restriction: $r_{2h} = R r_h$, where the 2D full-weighting stencil is

$$R_{2D} = \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix},$$

and the 3D version is the corresponding tensor-product stencil

$$(Rr)_{I,J,K} = \frac{1}{64} \sum_{a,b,c=-1}^1 w_a w_b w_c r_{2I+a, 2J+b, 2K+c}, \quad w_0 = 2, \quad w_{\pm 1} = 1.$$

- Prolongation: $e_h = P e_{2h}$, where 2D uses bilinear interpolation and 3D uses trilinear interpolation; for example,

$$e_h(2I+1, 2J) = \frac{1}{2} (e_{2h}(I, J) + e_{2h}(I+1, J)), \quad e_h(2I+1, 2J+1) = \frac{1}{4} \sum_{p,q \in \{0,1\}} e_{2h}(I+p, J+q),$$

and the 3D case uses the analogous tensor-product trilinear interpolation, involving 2, 4, or 8 coarse-grid neighbors depending on the fine-grid point location.

- Coarsest-grid solve: exact solve by default, with an RB-SOR coarse-solve variant also available.

Transfer operators and cycles: V-cycle workflow

One V-cycle

- Pre-smooth on the current grid to damp high-frequency error.
- Compute the residual $r_h = h^2 f - A_h \phi$, where A_h is the dimensionless discrete Poisson stencil.
- Restrict the residual to the coarse grid: $r_{2h} = R r_h$.
- Solve the coarse-grid error equation recursively, $A_{2h} e_{2h} = r_{2h}$.
- Prolongate the correction and update the solution: $\phi \leftarrow \phi + P e_{2h}$.
- Post-smooth and repeat until the relative scaled residual satisfies $\eta \leq \text{tol}$.

pre-smooth $\rightarrow$ compute residual (r_h) $\rightarrow$ restrict ($r_{2h} = R r_h$)

$\rightarrow$ coarse solve ($A_{2h} e_{2h} = r_{2h}$) $\rightarrow$ prolongate/update ($\phi \leftarrow \phi + P e_{2h}$) $\rightarrow$ post-smooth

Transfer operators and cycles: W-cycle and grid hierarchy

W-cycle

- Start like a V-cycle: pre-smooth, form the residual, and restrict to the next coarser grid.
- Call the coarse level twice before coming back up to the fine grid.
- The repeated coarse visit reduces the remaining low-frequency error, but it also increases the per-cycle cost.

pre-smooth $\rightarrow$ compute residual (r_h) $\rightarrow$ restrict ($r_{2h} = Rr_h$)

$\rightarrow$ coarse cycle $\times 2 \rightarrow$ prolongate/update ($\phi \leftarrow \phi + Pe_{2h}$) $\rightarrow$ post-smooth

MG-compatible grid sizes

- `--grid-size` is the number of interior points per axis, so the full array size is `grid-size + 2`.
- Nested coarsening uses $n_c = (n_f - 1)/2$, and the recursion stops when $n \leq 4$.
- Valid families: $2^a - 1$, $3 \cdot 2^a - 1$, and $5 \cdot 2^a - 1$.

Coarse-grid solve and cost: Coarse-grid solver design

Why the bottom level matters

- The coarsest solve closes the multigrid hierarchy.
- A weak bottom-level solve can leak error back to the fine grid after prolongation.
- The choice affects both robustness and time-to-solution.

Our implementation

- Default: exact/direct solve on the coarsest grid.
- Optional: RB-SOR coarse solve for comparison or fallback.

Trade-off

- RB-SOR keeps the code path simple and is easy to benchmark.
- Exact/direct gives the strongest bottom-level correction when the coarse system is tiny.

Coarse-grid solve and cost: V- and W-cycle work estimates

Coarse-grid scaling

Assume the fine-grid work is N . One level down:

$$2D : N \rightarrow N/4, \quad 3D : N \rightarrow N/8$$

V-cycle

$$T_V \sim N + \frac{N}{2^d} + \frac{N}{2^{2d}} + \dots$$

$$T_V^{2D} \sim \frac{4}{3}N, \quad T_V^{3D} \sim \frac{8}{7}N$$

W-cycle

$$T_W \sim N + 2\frac{N}{2^d} + 4\frac{N}{2^{2d}} + \dots$$

$$T_W^{2D} \sim 2N, \quad T_W^{3D} \sim \frac{4}{3}N$$

W / V ratio and takeaway

$2D : T_W/T_V = 3/2 \approx 50\%$, $3D : T_W/T_V = 7/6 \approx 17\%$. 3D shrinks faster, so the extra W-cycle coarse-grid work matters less than in 2D.

Implementation optimizations: OMP vs CUDA MG

OMP

- 'parallel for' + 'simd' on the hot stencil loops.
- Focus on CPU core utilization and vectorization.
- Goal: make the smoother and transfer operators scale better on the host.

CUDA

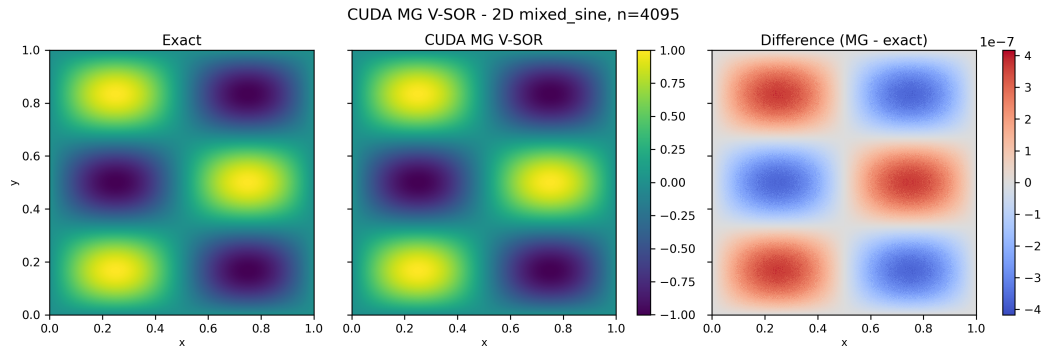
- Fuse residual and restriction to cut global memory traffic.
- Pool MG workspace and capture the iteration loop with CUDA Graph.
- Use fused small-grid kernels on coarse levels to avoid repeated launches.

Note

Red-black ordering is an enabler for parallel updates, not the main optimization itself.

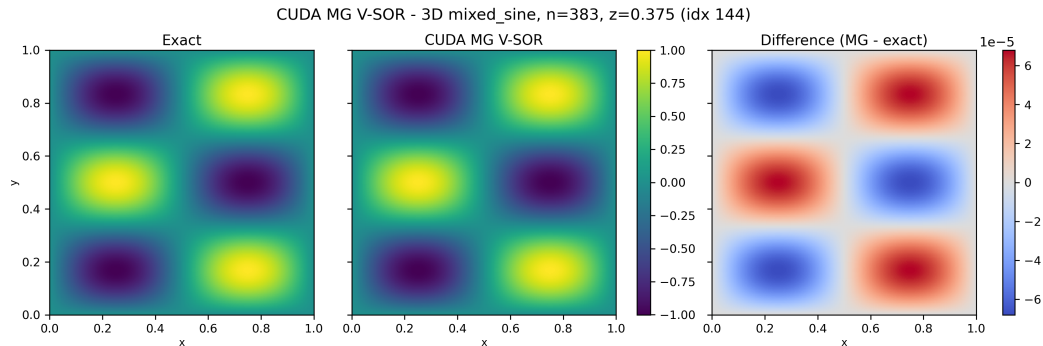
- 1 Problem Formulation
- 2 Multigrid Method
- 3 CUDA Multigrid Results**
- 4 OpenMP Performance
- 5 Conclusions

Mixed-sine test cases: 2D mixed-sine solution



CUDA MG V-SOR on the manufactured solution $\phi_{\text{exact}}(x, y) = \sin(2\pi x) \sin(3\pi y)$ with $\nu = 3$ and $\omega = 1.25$.

Mixed-sine test cases: 3D mixed-sine solution

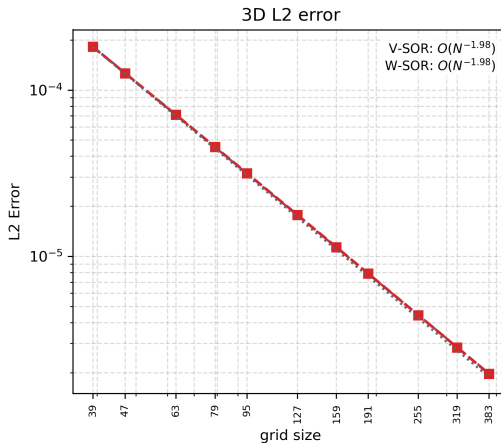
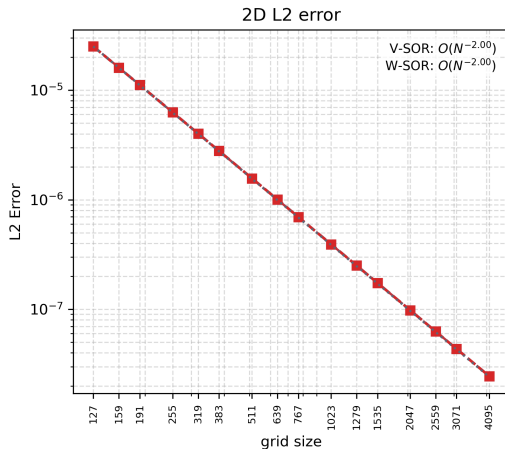


CUDA MG V-SOR on the manufactured solution $\phi_{\text{exact}}(x, y, z) = \sin(2\pi x) \sin(3\pi y) \sin(4\pi z)$ with $\nu = 3$ and $\omega = 1.25$.

Convergence analysis: L2 error convergence

CUDA MG convergence - sine (2D/3D L2 error)

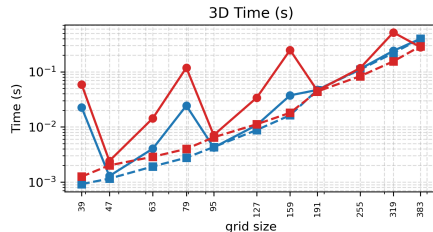
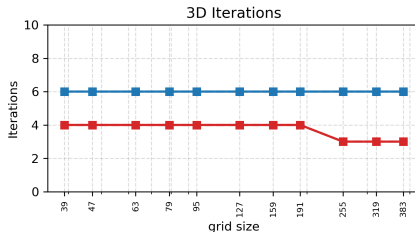
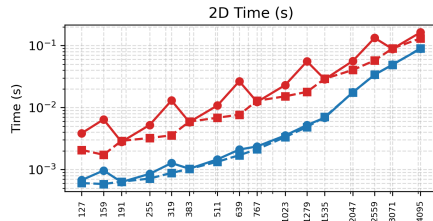
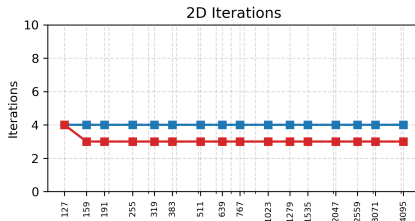
—■ V-SOR —■ W-SOR



Convergence analysis: Iteration and time vs exact solve

CUDA MG convergence - sine (all modes)

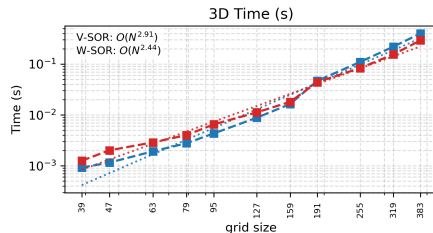
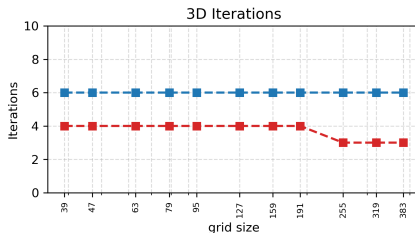
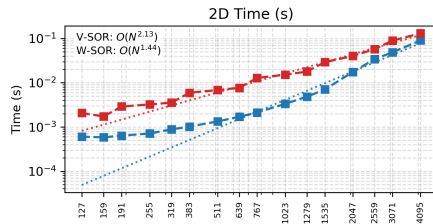
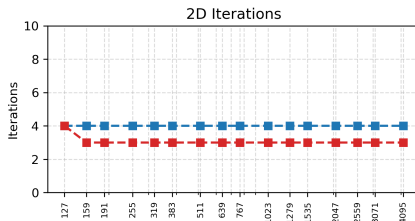
—●— V-exact —■— V-SOR —●— W-exact —■— W-SOR



Convergence analysis: Iteration-and-time fit

CUDA MG convergence - sine (SOR only)

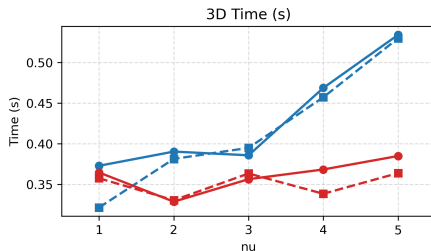
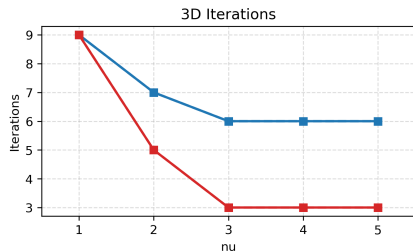
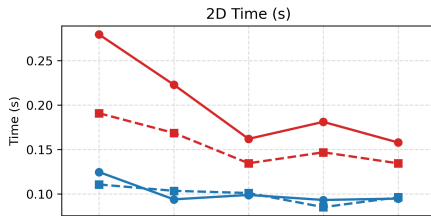
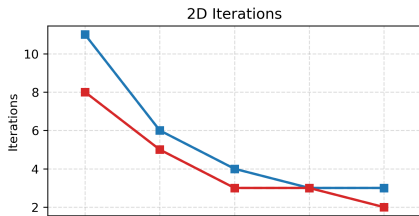
—■— V-SOR —■— W-SOR



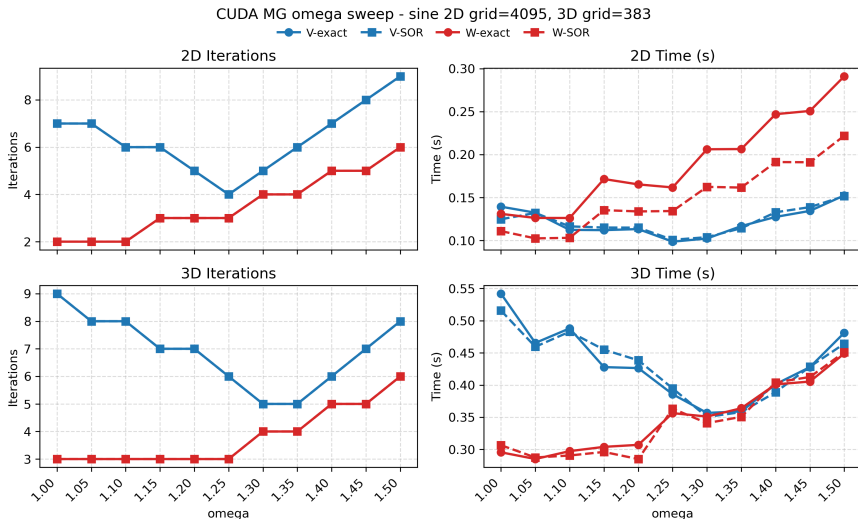
Parameter sweeps: Iteration and time vs ν

CUDA MG nu sweep - sine 2D grid=4095, 3D grid=383

—●— V-exact —■— V-SOR —●— W-exact —■— W-SOR



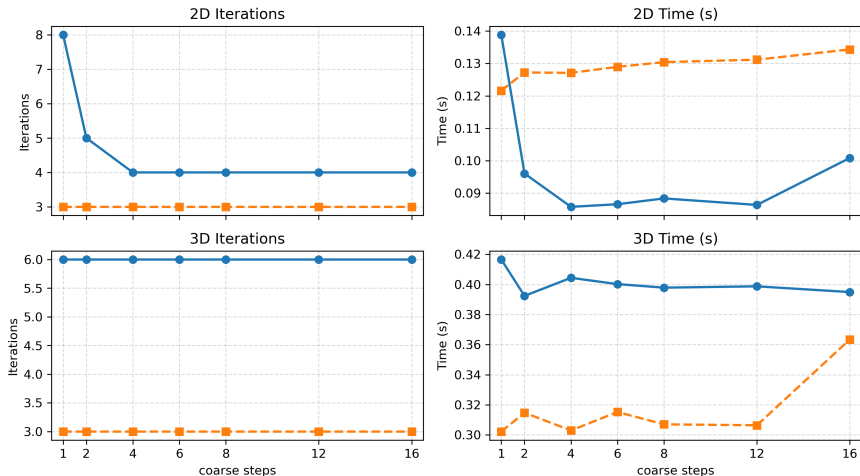
Parameter sweeps: Iteration and time vs ω



Parameter sweeps: Iteration and time vs coarse steps

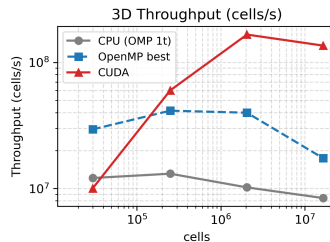
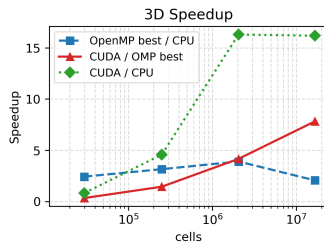
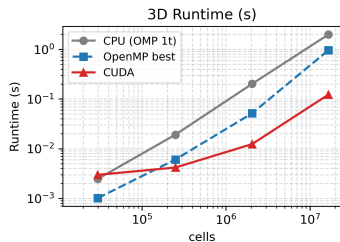
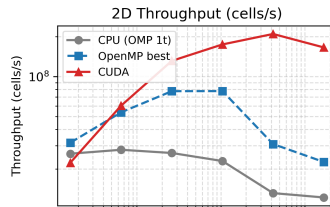
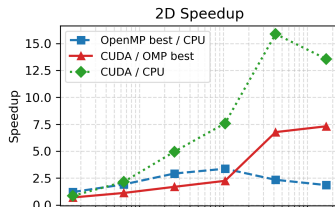
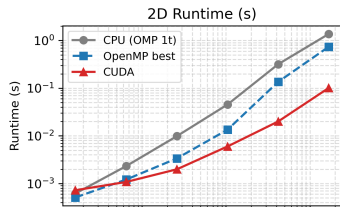
CUDA MG coarse_steps sweep - sine 2D grid=4095, 3D grid=383

● V-SOR ■ W-SOR



Performance scaling: CUDA scaling

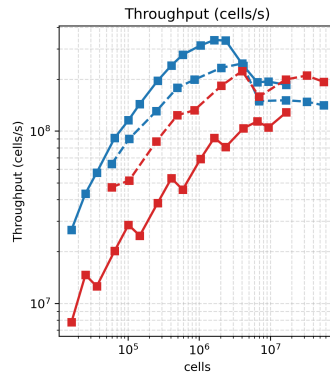
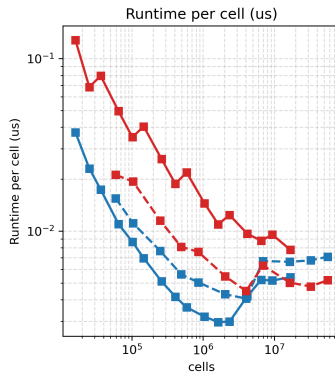
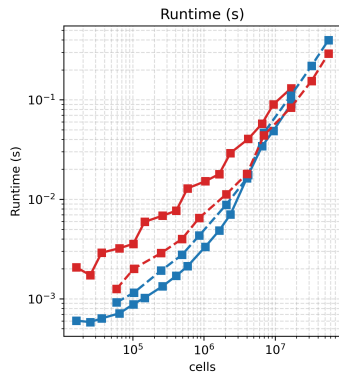
CUDA GPU scaling - sine
2D and 3D rows, MG V exact, CPU=OMP 1 thread, OpenMP=best thread, CUDA



Performance scaling: 2D vs 3D scaling

CUDA MG 2D vs 3D scaling - sine V/W-cycle, SOR coarse solve

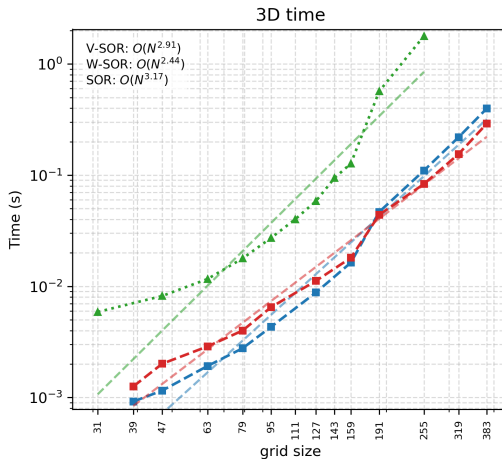
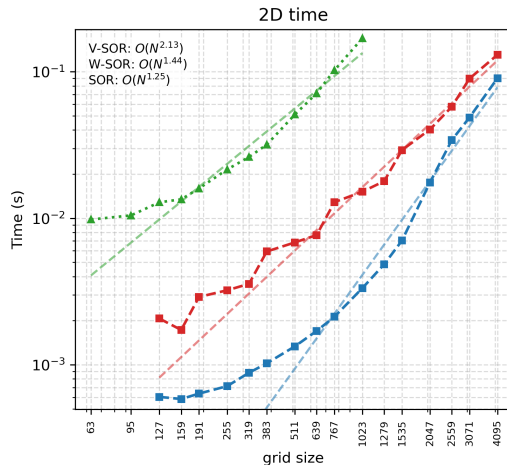
—■— 2D MG V SOR —■— 2D MG W SOR —■— 3D MG V SOR —■— 3D MG W SOR



Performance scaling: Multigrid vs SOR runtime

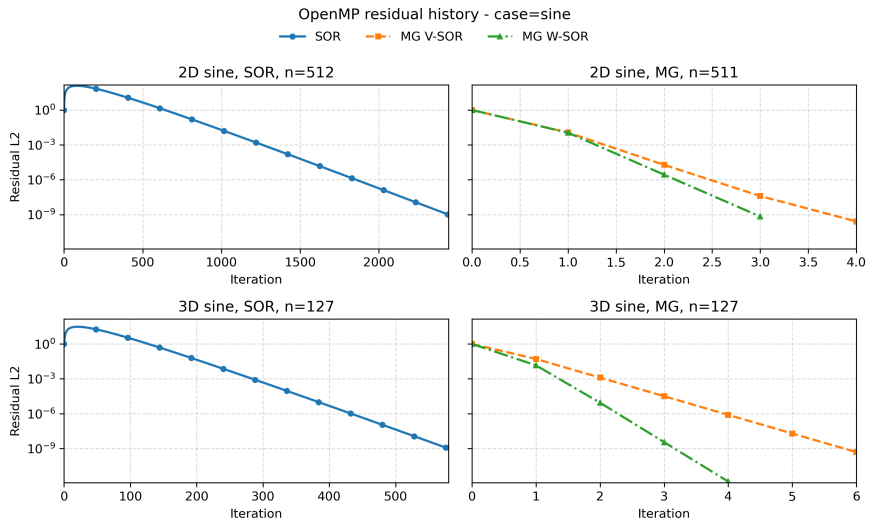
CUDA MG convergence vs CUDA SOR benchmark - sine

—■— V-SOR —■— W-SOR —●— SOR



- 1 Problem Formulation
- 2 Multigrid Method
- 3 CUDA Multigrid Results
- 4 OpenMP Performance**
- 5 Conclusions

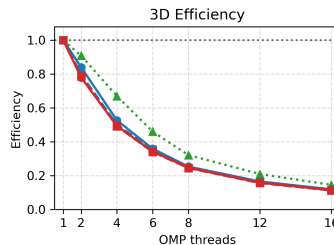
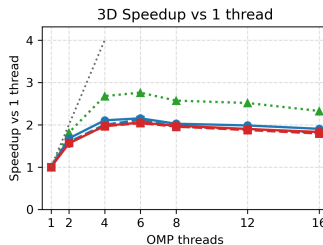
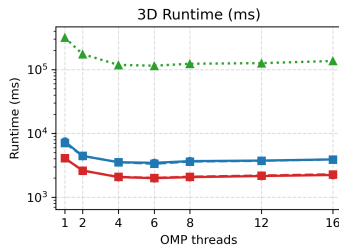
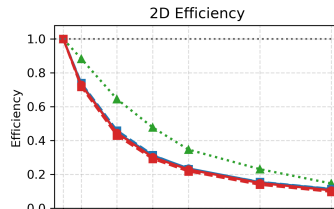
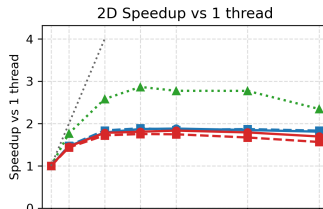
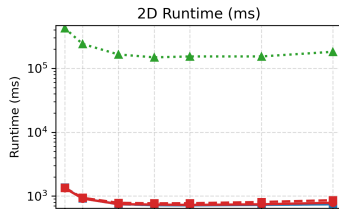
Empirical results: Residual history (OpenMP)



Empirical results: OpenMP thread scaling

OMP thread scaling - sine (2D grid=4095, 3D grid=383)

●▲ SOR ● MG V exact ■ MG V SOR ● MG W exact ■ MG W SOR



Bottleneck analysis: OpenMP bottleneck analysis

Topdown, L1, and cache evidence

- The OpenMP solver is backend-bound, not compute-bound.
- topdown-be-bound is 57.7% in 2D and 58.2% in 3D.
- topdown-mem-bound rises from 43.7% in 2D to 49.3% in 3D.
- L1D miss rate more than doubles, from 1.5% in 2D to 3.2% in 3D.
- The 3D LLC load miss rate is 89.4%, much higher than the 2D value of 26.4%.

Profile takeaway

- The hot kernel is `smooth_red_black / smooth_red_black_3d`.
- `perf annotate` shows the inner loop dominated by grid-value loads and rhs fetches.
- The kernel has low arithmetic intensity and spends most of its time waiting on memory.
- Conclusion: poor OpenMP thread scaling is mainly caused by memory bandwidth and cache pressure.

- 1 Problem Formulation
- 2 Multigrid Method
- 3 CUDA Multigrid Results
- 4 OpenMP Performance
- 5 Conclusions**

Conclusions: Conclusions and takeaways

What we learned

- The CUDA multigrid solver converges reliably in both 2D and 3D on manufactured-solution tests.
- The ν and ω sweeps show that solver time is sensitive to smoother tuning, especially in 2D.
- Compared with SOR, multigrid gives the better overall time-to-solution story as the grid grows.
- The OpenMP backend flattens because the hot smoothing kernel is memory-bound rather than compute-bound.

Bottom line

Multigrid is the right algorithmic direction here, CUDA gives the strongest performance path, and the remaining CPU scaling limit is mainly a memory-bandwidth problem.

Conclusions: References

- Numerical Recipes, 3rd ed., Section 20.6, *Multigrid Methods for Boundary Value Problems*.
- Achi Brandt and Oren E. Livne, *Multigrid Techniques: 1984 Guide with Applications to Fluid Dynamics, Revised Edition* (SIAM, 2011).
<https://epubs.siam.org/doi/book/10.1137/1.9781611970753>

Q&A

Thank you